

Git Flow: Strategic Analysis & Recommendations

Branching Strategy for Sprint-Based Releases and Preview-Validated Deployments

September 2026 | Kalderos LLC | REVISED

Contents

1. Executive Summary
2. Why Git Flow Fits Better
3. Git Flow Implementation
4. Git Flow vs GitHub Flow Comparison
5. Implementation Path
6. Alignment with Your Build/Deployment Process
7. Why This Works for Your Team
8. Next: kald-devops Integration

Executive Summary

After analyzing the actual build and deployment process, **Git Flow is the recommended strategic direction**, not GitHub Flow. Your process already follows Git Flow principles — the question is formalizing it:

- 1 Sprint-based release cycles (not continuous deployment)
- 2 Preview environment as pre-release validation gate
- 3 Hotfixes originating from Preview with mixed application versions
- 4 Coordinated deployment across 14+ applications
- 5 Client-specific deployment flows (you manage Gilead environments)

Why Git Flow Fits Better

Your Current Workflow (Already Git Flow-Like)

Your actual process already follows Git Flow principles:

Phase	Current Process	Git Flow Equivalent
Daily Development	Feature branches → Main	Feature branches → Develop
Sprint End	Main pushed to Dev/ Automated	Create release/* branch from develop
Pre-Release Testing	Artifacts deployed to Preview	release/* branch tested in Preview
Bug Found in Preview	Fixed, then tested again	Bugfixes committed to release/* branch
Release Ready	Merge to main, create tag	Merge release/* to main, create tag (no patch)
Production Deployment	Deploy from tag to Prod	Deploy fully-tested version to Prod
Production Bug (Rare)	Hotfix branches created	hotfix/* branches from main, merge back to main + develop

The insight: You're already doing Git Flow. The question is whether to formalize it.

Git Flow Implementation

Branch Structure

main (production releases only)

↑ (hotfixes merge back here)

develop (integration branch for next release)

↑ (features merge here)

feature/* (individual features, PRs to develop)

example: feature/payment-validation

release/* (pre-release validation, created from develop)

example: release/0.2.0

Tests in Preview environment before main merge

hotfix/* (production fixes, created from main)

example: hotfix/critical-bug-fix

Tests in Preview, then deployed to Prod

Workflow: Standard Release

- 1 **Development:** Features merge to `develop` throughout sprint
- 2 **Sprint End:** Create `release/0.2.0` branch from `develop`
- 3 **Deploy to Preview:** Artifacts from `release/*` go to Preview for QA testing
- 4 **Bug Found:** Fix directly in `release/*` branch (not in `develop`)
- 5 **Testing Passes:** Merge `release/*` to `main` and `develop`

- 6 **Tag:** Create v0.2.0 tag on main
- 7 **Deploy to Prod:** Build and deploy versioned artifacts
- 8 **Delete:** Remove release/* branch after merge

Workflow: Release Bugfix (Bug Found in Preview)

- 1 **Bug Found in Preview:** Analyze which apps are affected
- 2 **Fix in Release Branch:** Commit fix directly to `release/0.2.0`
- 3 **Fix:** Make changes (only affected apps)
- 4 **Test in Preview:** Deploy mixed versions (fixed + current stable)
- 5 **Testing Passes:** Release is ready
- 6 **Merge to Main:** `git merge release/0.2.0 --no-ff main`
- 7 **Tag Release:** `git tag v0.2.0` (no patch needed)
- 8 **Merge to Develop:** `git merge release/0.2.0 --no-ff develop`
- 9 **Delete:** Remove release/0.2.0 branch
- 10 **Deploy to Prod:** Deploy from v0.2.0 tag

Workflow: Hotfix (Production Bug After Release)

- 1 **Bug Found in Production:** Critical issue discovered after main tag
- 2 **Create Hotfix:** `git checkout -b hotfix/critical-payment-fix main`
- 3 **Fix:** Make changes (only affected apps)
- 4 **Test in Preview:** Validate with production data
- 5 **Merge to Main:** `git merge hotfix/* --no-ff main`
- 6 **Tag Patch:** `git tag v0.2.1` (patch version)
- 7 **Merge to Develop:** `git merge hotfix/* --no-ff develop`
- 8 **Delete:** Remove hotfix/* branch

9 **Deploy to Prod:** Deploy v0.2.1 patch

Git Flow vs GitHub Flow Comparison

Aspect	GitHub Flow	Git Flow (Recommended)
Release Cycle	Continuous (multiple per day)	Scheduled (weekly/sprint-based) ✓ Your Model
Persistent Branches	1 (main)	2 (main + develop) ✓ Better for coordination
Pre-Release Testing	No formal branch	release/* branches ✓ Your Preview environment
Release Bugfixes	No formal release branch	Fixed in release/* before main merge ✓ Prevents hotfixes
Production Hotfixes	Normal feature process to main	Dedicated hotfix/* branches ✓ For real production incidents
Testing Timing	After merge to main	Before merge to main ✓ Your Preview approach
Team Scaling	Works for small teams	Better for 20+ developers ✓ Your scale

Implementation Path

Immediate (Next Sprint)

- 1 Create and protect `develop` branch (same protection rules as main)
- 2 Set `develop` as default branch for developers
- 3 Document Git Flow workflow for team
- 4 Update CI/CD to build from both `develop` and `release/*` branches
- 5 Train team on `feature` → `develop` → `release/*` → `main` flow

Formalize Hotfix Process

Your `create-hotfix-branches.yml` workflow is good, but Git Flow naming makes it clearer:

- Use `hotfix/` prefix consistently
- Merge back to both `main` AND `develop` (not just one)
- Use `--no-ff` flags for clarity in history

Alignment with Your Build/Deployment Process

Git Flow Phase	Your Environment	Mapped Process
feature/* branches	Dev environment	Developers test locally before PR
develop branch	Dev + Automated	Latest development, automatically deployed
release/* branch	Preview environment	Pre-release validation with mixed versions
main branch	Prod environment	Only tagged releases, production deployments
hotfix/* branch	Preview (testing) → Prod	Emergency fixes validated in Preview first

Why This Works for Your Team

- **✓ Matches your actual process:** You're already doing Git Flow, just not formalizing it
- **✓ Handles complexity:** 14+ apps, mixed versions, client-specific flows all manageable
- **✓ Scales with team:** Works as team grows beyond 20 developers
- **✓ Clear releases:** No more v0.1.32.1, v0.1.32.2 proliferation
- **✓ Preview validation:** Testing before production tag (not after)
- **✓ Hotfix coordination:** Clear branch naming and merge strategy
- **✓ Audit trail:** Better git history for compliance

Next: kald-devops Integration

With Git Flow + your build/deployment process:

- `kald-devops list_pipelines --json` shows which versions are in each environment
- Develop branch = latest code for Dev/Automated
- Release/* branch = Preview environment artifacts
- Main tags = Production releases

This provides complete visibility via kald-devops CLI into your branching + deployment state.